Learning outcomes:

- Understanding Abstract Classes and Interfaces

Exercise 1:

Try to solve the interface challenge (the last part of the previous lecture)

Problem 2:

Implement a help desk ticketing system to manage help requests and support personnel. Help requests are categorized, and personnel respond according to their point levels.

Available categories (enum structure):

- InformationTechnologies
- HumanResources
- Accounting
- Sales
- Marketing
- Legal

By Using the Help.java file, implement the following classes:

1. *Employee* class
    - Properties: *FullName* (string), *PointLevel* (int), *AssignedCategories* (List)
    - Constructor to initialize all properties
2. *Ticket* class
    - Properties: *Id* (int), *Name* (string), *Category* (Category), *Point* (int), *AssignedEmployee* (string), *IsCompleted* (bool)
    - The constructor initializes the first 4 properties
3. *HelpDesk* class
    - Properties: *Employees (List), Tickets (List)*
    - Methods:
        - *AddTicket:* Add a ticket to the Tickets list
        - *AddEmployee:* Add an employee to the *Employees* list
        - *CompleteTicket:* Mark a ticket as completed if conditions are met
        - *GetWaitingTicketCount:* Return the count of incomplete tickets
        - *GetCompletedTicketsTotalPoint:* Return sum of points for completed tickets
        - *GetTicketsTotalPointByCategory:* Return list of (category, total points) tuples
        - *GetTicketsTotalPointByEmployee:* Return list of (employee name, total points) tuples
4. Create a main class to test the implemented methods, try to use a menu.

Example

Employees:

- John Doe (point level 2)
- Jane Roe (point level 5)

Tickets:

1. InformationTechnologies, 3 points
2. HumanResources, 1 point
3. Legal, 3 points
4. Sales, 10 points
5. InformationTechnologies, 1 point

Queries:

- John Doe, Ticket 1: Not completed (point level too low)
- John Doe, Ticket 2: Completed
- Jane Roe, Ticket 3: Completed
- John Doe, Ticket 5: Completed

Output:

```
WaitingTicketCount:2
CompletedTicketsTotalPoint:5
TicketsTotalPointByCategory:
InformationTechnologies:4
HumanResources:1
Accounting:0
Sales:10
Marketing:0
Legal:3
TicketsTotalPointByEmployee:
John Doe:2
Jane Roe:3
```

Input Format For Custom Testing:

The first line contains an integer n, the number of employees.
Each of the next n lines contains the employee information (FullName, PointLevel, AssignedCategories).
The next line contains an integer m, the number of tickets.
Each of the next m lines contains the ticket information (Id, Name, Category, Point).
The next line contains an integer k, the number of CompleteTicket calls.
Each of the next k lines contains the process for ticket completion (Employee FullName, Ticket Id).

Sample Case:

Sample Input For Custom Testing

```
2
John Doe,1,HumanResources InformationTechnologies
Jane Cherry,6,Legal InformationTechnologies
3
1,Ticket1,InformationTechnologies,1
2,Ticket2,HumanResources,5
3,Ticket3,Legal,2
4
John Doe,3
John Doe,1
```

```
Jane Cherry,2
Jane Cherry,3
```

Sample Output

```
WaitingTicketCount:1
CompletedTicketsTotalPoint:3
TicketsTotalPointByCategory:
InformationTechnologies:1
HumanResources:5
Accounting:0
Sales:0
Marketing:0
Legal:2
TicketsTotalPointByEmployee:
John Doe:1
Jane Cherry:2
```

Explanation

John Doe,1,Legal Sales -> Employee full name, *PointLevel*, and *AssignedCategories*, comma-separated. The categories are separated by a space.
2,Ticket1,HumanResources,7 -> Ticket Id, name, category, and points separated by a comma.
John Doe,4 -> Employee full name and ticket id separated by a comma.
There are 2 employees and 3 tickets.

- John Doe is shown tickets 3 and 1. Ticket 3 is for Legal, and he does not cover that department. Ticket1 is from a department he covers, and he has enough points.
- Jane Cherry sees tickets 2 and 3. Ticket2 is for HumanResources, and she does not cover that department. Ticket3 is from Legal, a department she covers, and she has enough points.
- Ticket2 remains unaddressed.

Problem 3:

Create a simple interface that allows an object to be saved to some sort of storage medium.

Implement the following:

1. ISaveable (interface)

- ✓ t has two methods:
  - `write()`, takes no arguments and returns a `List` containing objects of type `String`.
  - `read()`, takes a `List` of type `String` and doesn't return anything.

2. Player (class)

- ✓ It has four fields. Two `String`s called `name` and `weapon`. Two `int`s called `hitPoints` and `strength`.

- ✓ A constructor that accepts a `String` (`name`) and two `int`s (`hitPoints` and `strength`). It initialises `name`, `hitPoints` and `strength` with the newly passed in values. It initialises `weapon` with the default weapon `"Sword"`.
- ✓ And eleven methods:
  - Getters and setters for all four fields.
  - `write()`, same as interface. Return a `List` of the fields in the order they appear in `toString()`.
  - `read()`, same as interface. Store the values in the `List`, in the order they appear in `toString()`. Make sure the `List` is not `null` and the `size()` is greater than `0` before storing the values.
  - `toString()`, `Player`s *overriding* toString() method. It takes no arguments and returns a `String` in the following format:

```
Player{name='Tim', hitPoints=10, strength=15, weapon='Sword'}
```

3. Monster (class)

- ✓ It has three fields. One `String` called `name` and Two `int`s called `hitPoints` and `strength`.
- ✓ A constructor that accepts a `String` (`name`) and two `int`s (`hitPoints` and `strength`). It initialises `name`, `hitPoints` and `strength` with the newly passed in values.
- ✓ And six methods:
  - Only getters for the three fields.
  - `write()`, same as interface. Return a `List` of the fields in the order they appear in `toString()`.
  - `read()`, same as interface. Store the values in the `List`, in the order they appear in `toString()`. Make sure the `List` is not `null` and the `size()` is greater than `0` before storing the values.
  - `toString()`, Monsters *overriding* toString() method. It takes no arguments and returns a `String` in the following format:

```
Monster{name='Werewolf', hitPoints=20, strength=40}
```

TIP:

- `Player` and `Monster` need to implement `ISaveable`.
- Be extremely careful with the spelling of the names of the fields, constructors and methods.
- All fields are **private**.

Problem 4:

Implement the following:

1. ListItem (abstract class)

- ✓ It has three `protected` fields. Two `ListItem`s called `rightLink` and `leftLink`, and an `Object` called `value`.

- ✓ A constructor that takes an `Object` and initialises the field `value` with the parameter that was passed in.
- ✓ And seven methods:
    - `next()`, `setNext()`, `previous()`, `setPrevious()` and `compareTo()` which are `package-private` and `abstract` (see child class for declaration).
    - `getValue()`, takes no parameters and returns its `value`.
    - `setValue()`, takes an `Object` and assigns it to `value`.

2. Node (concrete class)

- ✓ It extends `ListItem`.
- ✓ It has a constructor that takes an `Object`, then calls its parent constructor with the parameter that was passed in.
- ✓ And five methods which are `package-private`:
    - `next()`, takes no parameters and returns the `ListItem` to its right.
    - `setNext()`, takes a `ListItem` and sets it as its `rightLink`, then it returns `rightLink`.
    - `previous()`, takes no parameters and returns the `ListItem` to its left.
    - `setPrevious()`, takes a `ListItem` and sets it as its `leftLink`, then it returns `leftLink`.
    - `compareTo()`, takes a `ListItem` and compares it to the `ListItem` that called this method. Use `value` from `ListItem` for comparison. If this `value` is greater than the `value` that was passed in, then it should return a number greater than zero. If vice versa, then it should return a number less than zero, and zero if equal.

3. MyLinkedList (concrete class)

- ✓ It implements `NodeList`.
- ✓ It has one field of type `ListItem` called `root`.
- ✓ A constructor that takes a `ListItem` and initialises the field `root` with the newly passed in parameter.
- ✓ And four methods:
    - `getRoot()`, getter for `root`.
    - `addItem()`, takes a `ListItem` and returns `true` if it was added successfully or `false` otherwise. If the item is already present, it doesn't get added. Use `compareTo()` to place the item in its proper order.
    - `removeItem()`, takes a `ListItem` and returns `true` if it was removed successfully or `false` otherwise.
    - `traverse()`, takes the `root` as an argument and does not return anything. If the `root` is `null` it prints out: `The list is empty`, otherwise print each value on a separate line.

4. NodeList (interface)

- ✓ It has four methods:

- `getRoot()`, `addItem()`, `removeItem()` and `traverse()` which are `package-private` and `abstract`

5. SearchTree (concrete class)

- ✓ It implements `NodeList`.
- ✓ It has one field of type `ListItem` called `root`.
- ✓ A constructor that takes a `ListItem` and initialises the field `root` with the newly passed in parameter.
- ✓ And five methods:

    - `getRoot()`, getter for `root`.
    - `addItem()`, similar to `MyLinkedList`. See second **TIP** below.
    - `removeItem()`, same as `MyLinkedList`.
    - `performRemoval()`, takes two `ListItem`s, the item to be removed and its parent. It doesn't return anything and is declared as `private`. Call this method from `removeItem()` when the item is found.
    - `traverse()`, takes the `root` as an argument and does not return anything. It uses *recursion* to visit all the branches in the tree (Inorder). Print each value on a seperate line.

TIP:

- The rules for adding an item to the linked tree are:
- If the head of the tree is null, make the head refer to the item to be added.
- If the item to be added is less than the current item in the tree, add the item before the current item (i.e., make the previous item's "next" refer to the new item, and the new item's "next" refer to the current item).
- If the item to be added is greater than the current item, move onto the next item and compare again (if there is no next item, then that is where the new item belongs).

TIP:

- When adding items to a Binary Search Tree, if the item to be added is less than the current item, then move to the left.
- If it is greater than the current item, then move to the right.
- The new item is added when an attempt to move in the required direction would involve following a null reference.
- duplicates are not allowed.

TIP:

- Inorder = print the **previous** node, then the **parent** node, and then the **next** node (left -> node -> right).